

UNIVERSIDAD DE ANTIOQUIA
DEPARTAMENTO DE ELECTRÓNICA Y TELECOMUNICACIONES
2598521 – INFORMÁTICA II



udea.png

Proyecto Final – JUEGO

Informe de Análisis y Diseño Técnico

Rigoberto Berrio Berrio
Estudiante de Ingeniería Electrónica
rigoberto.berrio1@udea.edu.co

Felipe Herrera Lopera
Estudiante de Ingeniería Electrónica
felipe.herreral@udea.edu.co

7 de junio de 2026

Índice

1. Introducción	2
2. Planteamiento del Problema	2
3. Definición General y Objetivos	2
3.1. Descripción general	2
3.2. Objetivos	2
3.3. Alcance	2
4. Especificación de Requerimientos	2
4.1. Requerimientos funcionales	2
4.2. Requerimientos no funcionales	3
4.3. Restricciones y dependencias	3
4.4. Usuarios destinatarios	3
5. Metodología y Planificación	3
5.1. Metodología empleada	3
5.2. Fases del desarrollo	3
5.3. Cronograma estimado	3
6. Diseño y Arquitectura del Sistema	3
6.1. Visión general	3
6.2. Descripción de los componentes principales	3
6.3. Diagrama general de clases	4
6.4. Descripción individual de clases	6
6.5. Dependencias externas y bibliotecas	8
7. Desarrollo e Implementación	8
7.1. Estructura general del código	8
7.2. Mecánicas del Nivel 1	8
7.3. Motor de físicas del Nivel 2	8
7.4. Inteligencia artificial	8
7.5. Patrón Singleton para recursos	8
7.6. Manejo de excepciones	8
8. Procedimientos de Prueba	8
8.1. Estrategia de verificación	8
8.2. Casos de prueba	9
8.3. Errores encontrados y corregidos	9
8.4. Revisión de calidad	9
9. Conclusiones	9

1. Introducción

Este informe documenta el análisis y diseño del proyecto final de la asignatura Informática II, en el que se desarrolló un videojuego de balonmano / fútbol utilizando el lenguaje C++ junto con el *framework* Qt. La idea surgió como una forma de aplicar los conceptos de programación orientada a objetos (POO) vistos durante el semestre, tales como herencia, polimorfismo, encapsulamiento y el uso de interfaces, en un contexto práctico y entretenido.

El juego cuenta con dos niveles de dificultad y un sistema de menú principal que permite al usuario navegar entre las diferentes opciones. En el **Nivel 1** el enfoque es más arcade: el jugador controla un personaje con una mecánica simple de físicas proyectiles y debe anotar goles evitando enemigos con distintos patrones de movimiento. En el **Nivel 2** la experiencia se vuelve más completa, incorporando inteligencia artificial, un motor de colisiones, gestión de audio, animaciones por *sprites* y un HUD que muestra el marcador y el tiempo restante.

El **público objetivo** de este software son estudiantes universitarios o personas jóvenes que disfruten de juegos casuales de deportes, aunque su propósito principal sigue siendo académico: demostrar el manejo de C++ y Qt en un proyecto estructurado.

2. Planteamiento del Problema

En la asignatura Informática II se exige como proyecto integrador la creación de un software de mediana complejidad que ponga en práctica los pilares de la POO. El reto consiste en ir más allá de los ejercicios básicos de clase y construir algo que tenga lógica de negocio real, interacción con el usuario y un diseño de clases bien organizado.

Un videojuego cumple con todos estos criterios porque obliga a resolver problemas concretos: ¿cómo se mueve un personaje?, ¿cómo detecta el sistema que dos objetos chocaron?, ¿cómo se decide el comportamiento de un enemigo? Cada una de estas preguntas lleva directamente al uso de herencia, interfaces, patrones de diseño y estructuras de datos, que son precisamente los temas centrales del curso.

Adicionalmente, Qt proporciona herramientas gráficas robustas para dibujar escenas, capturar eventos del teclado, reproducir sonidos y gestionar temporizadores, lo que permite concentrarse en la lógica del juego sin necesidad de escribir desde cero el motor gráfico.

3. Definición General y Objetivos

3.1. Descripción general

ProyectoFinal es un videojuego de escritorio desarrollado en C++17 con Qt 6. Consta de un menú principal y dos niveles jugables, cada uno con mecánicas distintas pero bajo una arquitectura de clases coherente. El ejecutable corre en sistemas operativos con soporte para Qt (Windows, Linux, macOS).

3.2. Objetivos

- Aplicar los conceptos de POO en un sistema con más de veinte clases interrelacionadas.
- Implementar al menos dos niveles de juego con mecánicas diferenciadas.
- Diseñar un sistema de inteligencia artificial básico para los enemigos del Nivel 2 que aprenda del comportamiento del jugador.
- Integrar un motor de físicas sencillo basado en cinemática (movimiento circular, proyectil y colisiones elásticas).
- Gestionar recursos multimedia (audio y *sprites*) de forma centralizada mediante el patrón *Singleton*.
- Elaborar la documentación técnica completa del proyecto.

3.3. Alcance

El proyecto abarca el diseño, implementación y prueba del videojuego, incluyendo el menú de inicio, los dos niveles, el sistema de puntuación, el HUD y el manejo de audio. No se contempla publicación en tiendas de aplicaciones ni modo multijugador en red.

4. Especificación de Requerimientos

4.1. Requerimientos funcionales

1. El sistema debe mostrar un menú principal que permita seleccionar el nivel de juego o salir de la aplicación.
2. El Nivel 1 debe permitir al jugador moverse, disparar el balón con física de proyectil y recoger un ítem (*jeringa*) que active una mejora temporal de velocidad.
3. El Nivel 1 debe generar enemigos con cuatro patrones de movimiento distintos: horizontal, vertical, circular y trayectoria en L.
4. El Nivel 2 debe soportar dos jugadores humanos (control por teclado) con cambio de jugador activo.
5. El Nivel 2 debe incluir tres tipos de enemigos con IA que orbitan, persiguen, interceptan o regresan según el estado del juego.
6. Ambos niveles deben llevar registro del puntaje y mostrar el resultado al finalizar la partida.
7. El sistema debe reproducir efectos de sonido para los eventos principales del juego (gol, silbato, acciones

del enemigo).

8. Al terminar un nivel, el sistema debe dar opción de reiniciar o volver al menú.

4.2. Requerimientos no funcionales

- **Rendimiento:** el bucle de juego debe correr a al menos 60 fps en hardware de gama media.
- **Usabilidad:** los controles del teclado deben ser intuitivos y estar indicados en pantalla.
- **Mantenibilidad:** cada clase debe tener responsabilidad única y estar separada en su propio par de archivos `.h` / `.cpp`.
- **Portabilidad:** el código debe compilar sin modificaciones en cualquier sistema con Qt 6 instalado.
- **Escalabilidad:** la arquitectura debe permitir agregar nuevos niveles o tipos de enemigos sin reescribir el núcleo del sistema.

4.3. Restricciones y dependencias

- **Lenguaje:** C++17 estándar.
- **Framework:** Qt 6 (módulos Widgets, Multimedia, Core).
- No se permite el uso de motores de juego externos (Unity, Godot, etc.).
- Los recursos gráficos y de audio deben ser de libre uso o creación propia.

4.4. Usuarios destinatarios

El único tipo de usuario es el **jugador**, que interactúa con la aplicación mediante teclado. No existe un perfil de administrador ni acceso a base de datos.

5. Metodología y Planificación

5.1. Metodología empleada

Para el desarrollo del proyecto se utilizó una metodología **iterativa e incremental**, cercana a los principios ágiles pero adaptada al contexto académico del semestre. En lugar de entregar todo al final, el trabajo se organizó en ciclos cortos donde al terminar cada uno había algo funcional que se podía probar y revisar.

Esto fue útil porque permitió detectar errores de diseño temprano. Por ejemplo, en las primeras iteraciones el sistema de colisiones estaba mezclado dentro de las clases de los personajes; luego de revisarlo se separó en una clase **PhysicsEngine** independiente, lo que hizo el código mucho más limpio.

5.2. Fases del desarrollo

1. **Fase 1 – Análisis y diseño inicial:** definición de clases, relaciones y boceto del diagrama UML. Duración aproximada: 2 semanas.
2. **Fase 2 – Menú y estructura base:** implementación del menú principal, el controlador de niveles y

la ventana principal. 1 semana.

3. **Fase 3 – Nivel 1:** jugador, balón, enemigos con distintos patrones, lógica de goles y vidas. 2 semanas.
4. **Fase 4 – Nivel 2:** entidades del juego, motor de físicas, IA de los enemigos y el portero, HUD, audio y sprites. 3 semanas.
5. **Fase 5 – Integración y pruebas:** unión de los dos niveles bajo el controlador, corrección de errores y ajustes de balanceo. 1 semana.
6. **Fase 6 – Documentación:** redacción del informe final. 1 semana.

5.3. Cronograma estimado

Fase	S1	S2	S3	S4	S5	S6-10
Análisis/Diseño	•	•				
Menú base		•				
Nivel 1			•	•		
Nivel 2				•	•	•
Pruebas						•
Documentación						•

Cuadro 1: Cronograma de desarrollo (semanas)

6. Diseño y Arquitectura del Sistema

6.1. Visión general

El sistema está dividido en tres módulos principales que reflejan las responsabilidades del juego:

- **Módulo Menú:** controla la navegación entre pantallas y actúa como punto de entrada de la aplicación.
- **Módulo Nivel 1:** contiene toda la lógica del primer nivel, con físicas de proyectil y enemigos de patrón fijo.
- **Módulo Nivel 2:** implementa el nivel más complejo, con IA adaptativa, físicas más realistas y gestión de recursos multimedia.

Sobre esta base de módulos existe también una **capa de servicios** compartida: **SpriteManager** y **AudioManager**, implementados como *Singletons*, proveen *sprites* animados y efectos de sonido a cualquier clase que los necesite sin crear instancias múltiples.

6.2. Descripción de los componentes principales

JuegoController es el director general del programa. Se encarga de crear los objetos de cada nivel según la selección del usuario y de conectar sus señales Qt con los eventos correspondientes (pasar de nivel, reiniciar, volver al menú, etc.).

MainMenu es la pantalla de inicio. Escucha las teclas del jugador y emite señales Qt hacia **JuegoController** para indicar qué acción se desea realizar.

Nivel1 es el corazón del primer nivel. Contiene una lista de enemigos, el balón, el jugador y la portería.

En cada ciclo del temporizador actualiza la posición de todos los objetos, detecta colisiones manualmente y decide si se anota un gol o se pierde una vida.

GameEntity es la clase abstracta base de todas las entidades del Nivel 2. Define los atributos comunes (posición, velocidad, radio de colisión) y el método virtual puro `update(dt)`, que cada subclase implementa con su propia lógica.

Collidable es una interfaz (clase abstracta con solo métodos puros) que deben implementar todas las entidades capaces de colisionar. Esto permite que el motor de físicas trabaje con cualquier objeto sin conocer su tipo concreto.

AIAgent encapsula la lógica de decisión de los enemigos y del portero. Sigue un ciclo *percibir – razonar – actuar* y ajusta su velocidad máxima y tiempo de reacción a medida que avanza el partido (dificultad progresiva).

PhysicsEngine agrupa métodos estáticos para los cálculos físicos: posición circular, integración de Verlet, colisión elástica en 2D y fricción. Al ser estáticos, cualquier clase puede llamarlos sin necesidad de instanciar el motor.

Level2Scene es el escenario del Nivel 2. Orquesta todos los objetos de juego, corre el bucle principal a través de un `QTimer`, y delega la lógica de puntuación a `GameManager`.

6.3. Diagrama general de clases

La figura 1 presenta el diagrama UML completo del proyecto con las 24 clases identificadas, sus atributos, métodos y las relaciones entre ellas. Las relaciones representadas son: herencia (línea sólida + triángulo hueco), realización de interfaz (línea punteada + triángulo hueco), composición (línea sólida + rombo relleno), agregación (línea sólida + rombo vacío) y dependencia de uso (línea punteada + flecha abierta).

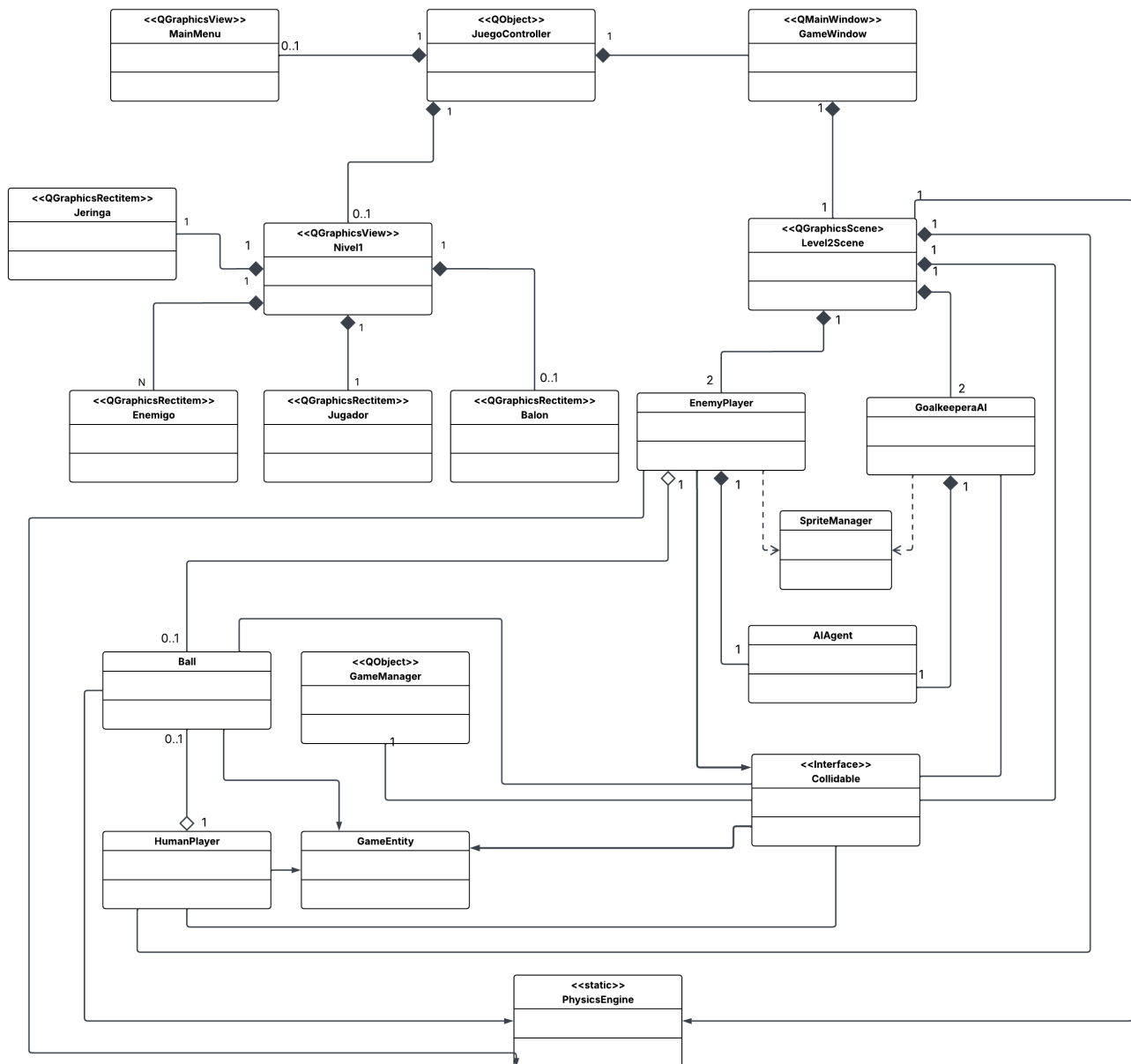


Figura 1: Diagrama de clases UML — ProyectoFinal

6.4. Descripción individual de clases

En la tabla 2 se presenta un resumen de cada clase, su módulo de pertenencia y su responsabilidad principal dentro del sistema.

Clase	Módulo	Responsabilidad
JuegoController	Menú	Controlador central; conecta menú y niveles mediante señales Qt.
MainMenu	Menú	Pantalla de inicio; emite la selección del nivel.
Jugador	Nivel 1	Personaje controlado por el usuario; maneja movimiento y disparo.
Enemigo	Nivel 1	Obstáculo con cuatro patrones de movimiento configurables.
Balon	Nivel 1	Proyectil con física parabólica; controla su propio ciclo de vida.
Jeringa	Nivel 1	Ítem de mejora que activa velocidad extra al ser recogido.
MenuInicio	Nivel 1	Pantalla de bienvenida del Nivel 1.
Nivel1	Nivel 1	Escenario del Nivel 1; coordina todos sus objetos.
MainWindow	Nivel 1	Ventana principal de la aplicación en el Nivel 1.
Vec2D	Nivel 2	Estructura de vector 2D con operaciones matemáticas.
GameEntity	Nivel 2	Clase abstracta base para todas las entidades del Nivel 2.
Collidable	Nivel 2	Interfaz que deben implementar los objetos con colisión.
Ball	Nivel 2	Balón del Nivel 2; maneja estados (libre, sostenido, disparado).
HumanPlayer	Nivel 2	Jugador humano; responde a eventos de teclado.
EnemyPlayer	Nivel 2	Enemigo del campo con IA y tres variantes (Taz, Bugs, Daffy).
GoalkeeperAI	Nivel 2	Portero con IA que aprende los patrones de disparo del rival.
AIAgent	Nivel 2	Motor de decisión IA: percibe, razona y actúa.
PhysicsEngine	Nivel 2	Colección de métodos estáticos para cálculos físicos.
HUD	Nivel 2	Interfaz en pantalla: marcador, tiempo, posesión y dificultad.
AudioManager	Nivel 2	Singleton que carga y reproduce efectos de sonido y música.
SpriteManager	Nivel 2	Singleton que carga y provee fotogramas de animación.
GameManager	Nivel 2	Controla el estado de la partida, tiempo y puntaje.
GameWindow	Nivel 2	Ventana principal del Nivel 2; gestiona pantallas de inicio/fin.
Level2Scene	Nivel 2	Escenario del Nivel 2; corre el bucle de juego y orquesta todo.

Cuadro 2: Resumen de clases del proyecto

6.5. Dependencias externas y bibliotecas

- **Qt 6 – Widgets:** proporciona `QGraphicsView`, `QGraphicsScene`, `QGraphicsItem` y las clases de ventana. Es la base visual de toda la aplicación.
- **Qt 6 – Core:** suministra el sistema de señales y *slots*, los temporizadores (`QTimer`), los punteros inteligentes (`QPointer`) y las cadenas (`QString`).
- **Qt 6 – Multimedia:** aporta `QSoundEffect` para reproducir efectos de audio de baja latencia.
- **Biblioteca estándar C++17:** se usan `std::vector`, `std::map`, `std::array` y `std::unique_ptr` para la gestión de colecciones y memoria.

No se utilizaron librerías de terceros adicionales; todo el desarrollo se realizó sobre las herramientas provistas por Qt y el estándar de C++.

7. Desarrollo e Implementación

7.1. Estructura general del código

El proyecto se organizó en dos carpetas principales (`nivel1/` y `nivel2/`) más una carpeta `menu/` para los archivos compartidos. Dentro de `nivel2/` existe un subdirectorío `include/` con todos los encabezados, lo que facilita la separación entre interfaz y lógica.

7.2. Mecánicas del Nivel 1

La física del balón se implementó con las ecuaciones de movimiento parabólico. En cada *tick* del `QTimer` (60 Hz) se incrementa el tiempo interno del balón y se recalcula su posición con:

$$x(t) = x_0 + v_{x_0} t, \quad y(t) = y_0 + v_{y_0} t + \frac{1}{2} g t^2$$

Cuando el tiempo supera un límite o el balón sale de la pantalla, se desactiva y el jugador recupera el control.

Los enemigos reciben en su constructor el tipo de movimiento que deben realizar. Al actualizarse, consultan ese tipo y aplican la fórmula correspondiente: movimiento armónico simple para los modos horizontal y vertical, y función paramétrica circular para el modo circular.

7.3. Motor de físicas del Nivel 2

`PhysicsEngine` implementa la **integración de Verlet**, que es más estable numéricamente que el método de Euler simple. La actualización de posición se calcula así:

$$\mathbf{p}_{n+1} = \mathbf{p}_n + \mathbf{v}_n \Delta t + \frac{1}{2} \mathbf{a}_n \Delta t^2$$

Para las colisiones entre entidades se usó el modelo de **colisión elástica en 2D**, calculando las velocidades resultantes a partir de las masas y las velocidades relativas en el eje normal al punto de impacto.

7.4. Inteligencia artificial

El ciclo de la IA sigue tres pasos en cada fotograma:

1. **Percibir:** el agente recibe las posiciones del balón, el jugador y su propia posición como parámetros.
2. **Razonar:** según la situación (¿tiene el balón el equipo rival?, ¿está el balón cerca de la portería?) se decide un estado: *orbitar*, *perseguir*, *interceptar* o *regresar*.
3. **Actuar:** se calcula un vector de dirección hacia el objetivo y se aplica una aceleración limitada por `maxSpeed`, que va incrementándose con el tiempo de juego para crear dificultad progresiva.

El portero además **aprende**: registra en qué zona (izquierda, centro, derecha) anotan más goles y ajusta su posición inicial de espera hacia esa zona en los siguientes disparos.

7.5. Patrón Singleton para recursos

`SpriteManager` y `AudioManager` siguen el patrón *Singleton* con instancia estática local:

```
static SpriteManager& instance() {
    static SpriteManager inst;
    return inst;
}
```

Esto garantiza que los recursos se carguen una sola vez en memoria y que cualquier clase pueda acceder a ellos sin pasar punteros por toda la jerarquía.

7.6. Manejo de excepciones

El Nivel 2 define una jerarquía propia de excepciones que hereda de `std::runtime_error`: `GameException` es la raíz, y de ella derivan `ResourceLoadException`, `InvalidGameStateException`, `OutOfBoundsException` y `PhysicsException`. Cada una se lanza en el contexto correspondiente para comunicar errores de forma clara y sin mezclar responsabilidades.

8. Procedimientos de Prueba

8.1. Estrategia de verificación

Las pruebas se realizaron de forma manual e incremental. Dado que el proyecto no incluye un *framework* de pruebas automatizadas (como Google Test), se optó por definir casos de prueba descriptivos que se ejecutaron durante el desarrollo de cada módulo.

8.2. Casos de prueba

Caso	Descripción	Resultado
CP-01	Jugador se mueve sin salirse de pantalla.	Aprobado
CP-02	Balón sigue trayectoria parabólica.	Aprobado
CP-03	Enemigos siguen sus cuatro patrones.	Aprobado
CP-04	Jeringa activa y desactiva velocidad extra.	Aprobado
CP-05	Gol incrementa el marcador.	Aprobado
CP-06	Cambio de jugador activo en Nivel 2.	Aprobado
CP-07	Enemigos transicionan entre estados IA.	Aprobado
CP-08	Portero bloquea y reproduce animación.	Aprobado
CP-09	HUD actualiza marcador y tiempo.	Aprobado
CP-10	Pantalla de fin aparece al agotar tiempo.	Aprobado
CP-11	Sonido de gol se reproduce al anotar.	Aprobado
CP-12	Sprites cambian de fotograma de forma fluida.	Aprobado

Cuadro 3: Casos de prueba ejecutados

8.3. Errores encontrados y corregidos

Durante las pruebas se detectaron algunos problemas relevantes. El primero fue que la colisión entre el balón y los enemigos en el Nivel 1 no siempre se detectaba cuando los objetos se movían muy rápido; se corrigió reduciendo el intervalo del temporizador y ajustando el radio de colisión. El segundo fue que el **SpriteManager** intentaba cargar los *sprites* antes de que Qt inicializara el sistema gráfico, lo que lanzaba una excepción en el arranque; se solucionó moviendo la carga al momento en que la ventana ya estaba visible.

8.4. Revisión de calidad

El código fue revisado entre los dos integrantes del equipo al terminar cada fase. Se verificó que cada clase tuviera una responsabilidad clara, que no hubiera punteros sin inicializar y que los destructores liberaran los recursos correctamente. También se comprobó que el juego no presentara *memory leaks* visibles durante sesiones largas de juego.

9. Conclusiones

El desarrollo de este videojuego resultó ser una experiencia muy enriquecedora para afianzar los conceptos de programación orientada a objetos. Tener que diseñar más de veinte clases con relaciones claras entre ellas obliga a pensar con cuidado en la responsabilidad de cada componente y en cómo deben comunicarse, algo que es mucho más difícil de ver en ejercicios pequeños de clase.

Uno de los aprendizajes más importantes fue entender cuándo usar herencia y cuándo preferir composición. Al principio se tendía a heredar para reutilizar código, pero en varios casos resultó mejor que una clase *tuviera* otra en lugar de *ser* otra. Por ejemplo, los enemigos y el portero del Nivel 2 no heredan de **AIAgent** sino que lo contienen, porque la IA es un comportamiento que podría cambiar o intercambiarse sin alterar la entidad.

El patrón *Singleton* para **AudioManager** y **SpriteManager** demostró ser una solución práctica para recursos que deben cargarse una sola vez. Sin embargo, también se aprendió que abusar de este patrón puede hacer el código difícil de probar, por lo que debe usarse con criterio.

El motor de físicas basado en integración de Verlet fue más estable que el método de Euler que se intentó inicialmente, lo cual fue un buen ejemplo de cómo las decisiones técnicas tienen impacto directo en la calidad de la experiencia del usuario.